

Resumo do Seminário: Programa para Visualização de Grafos

Eduardo Augusto Duarte de Castro Giovannetti¹, Guilherme Carlos Freitas Bartasson²

¹ Prédio de Ciências da Computação
IF Goiano – Campus Rio Verde, Rodovia Sul Goiana - Km 1
Zona Rural - Rio Verde - GO – Brasil.
CEP: 75.901-970 - Caixa Postal 66

{eduardo.giovannetti, guilherme.bartasson}@estudante.ifgoiano.edu.br

Abstract. *This meta-paper presents a summary of a seminar presentation on the creation of an interactive graphical interface for graph visualization using Dash and the NetworkX library. It explains the underlying theoretical foundations of graph visualization alongside the implementation details of the control dashboard and the required Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms. Furthermore, it incorporates the mapping and identification of Strongly Connected Components (SCC) within complex cyclic digraphs. The results demonstrate how the application of interactive visual features and execution snapshots enhances the comprehension of abstract structural properties, bridging the theoretical fundamentals of graph theory with software development practices.*

Resumo. *Este meta-artigo apresenta o resumo da apresentação de seminário sobre a criação de uma interface gráfica interativa para visualização de grafos utilizando o Dash e a biblioteca NetworkX. Explica-se, no processo, a base teórica por trás dessa visualização e as implementações feitas para o painel de controle e para os algoritmos exigidos de busca em largura (BFS) e busca em profundidade (DFS). Adicionalmente, incorpora-se o mapeamento e a identificação de Componentes Fortemente Conexas (SCC) em digrafos cíclicos complexos. Os resultados demonstram como a aplicação de recursos visuais interativos e snapshots de execução enriquece o entendimento de propriedades estruturais abstratas, unindo os fundamentos teóricos da teoria dos grafos à prática do desenvolvimento de software.*

1. Introdução

A teoria dos Grafos é um ramo da matemática discreta que estuda as relações entre elementos de um conjunto, representadas por estruturas chamadas grafos, compostas por vértices e arestas. Atualmente, é amplamente aplicada em diversas áreas, especialmente em Ciência da Computação, onde é fundamental para algoritmos como Dijkstra (caminho mais curto), BFS (Busca em Largura), e DFS (Busca em Profundidade).

Com isso em mente, este artigo busca apresentar sobre um programa, desenvolvido em Python, que permite a visualização junto com a interação de tais estruturas, além da execução correta dos algoritmos de busca BFS e DFS. Será abordado também mais sobre essa base teórica a fim de introduzir os conceitos implementados no visualizador.

1.1. Grafos: Conceitos Iniciais

Em teoria dos grafos, um grafo G é um **par ordenado** $(V(G), E(G))$ consistindo de um conjunto $V(G) = (v_1, v_2, \dots, v_n)$ de vértices e um conjunto $E(G) = (e_1, e_2, \dots, e_m)$ de arestas. Cada **aresta** consiste de um par, não necessariamente distinto, de vértices de G chamados extremos ou pontas finais.

Um **passeio** em G é uma sequência finita e não vazia $P = (v_0, e_0, v_1, e_1, \dots, e_i, v_k)$, tal que, $e_i = (v_k, v_k + 1)$ para todo $i < k$, $(v_i, v_i) \in E(G)$. Se os vértices no passeio P são distintos, P é chamado um **caminho**. Um **caminho** é simples se não repete vértices, exceto possivelmente o primeiro e o último em um ciclo. Alguns conceitos importantes para estrutura de um grafo são:

- **Grau de um vértice:** Em um grafo não orientado, o grau de um vértice é o número de arestas incidentes a ele;
- **Graus de entrada e grau de saída:** Em um grafo orientado, o grau de entrada é o número de arestas que chegam a um vértice v , enquanto o grau de saída é o número de arestas que saem de um vértice v ;
- **Vizinhança:** A vizinhança de um vértice v em um grafo G refere-se ao conjunto de vértices adjacentes a v , ou ao subgrafo induzido por esses vértices;
- **Laço:** Os laços em um grafo é uma aresta que conecta um vértice a ele mesmo. Em grafos não direcionados, um laço contribui com dois para o grau do vértice, pois é contado duas vezes — uma para cada extremidade da aresta, que coincide com o próprio vértice. Em grafos direcionados, um laço adiciona um ao grau de entrada e um ao grau de saída do vértice;

1.2. Classificações

Um grafo G é definido tanto por seus elementos básicos (vértices e arestas) quanto pelas características globais que determinam sua estrutura. Ao analisar o grafo G em sua totalidade, destacam-se algumas propriedades e classificações fundamentais, tais como:

- **Orientação:** Quanto a orientação um grafo pode receber a classificação **Orientado** ou **Não Orientado**. Um digrafo, também conhecido como grafo orientado, é um tipo de grafo em que as arestas possuem uma direção, indicada por uma seta na representação gráfica. Isso significa que a relação entre dois vértices é unidirecional: uma aresta de v para w não implica a existência de uma aresta de w para v . Enquanto o **não orientado**, não possuem direção definida nas arestas, o que implica que se há uma aresta entre os vértices w e v , é possível ir de w para v e de v para w indistintamente.
- **Regularidade:** A classificação estrutural referente ao grau de cada vértice v pertencente ao grafo G , que define se o grafo é regular, irregular ou maximalmente não regular.
- **Completude (K_n):** Um grafo é K_n completo se para todo o par de vértices v está conectado por uma aresta e .
- **Bipartição:** Os vértices podem ser divididos em dois conjuntos, com arestas apenas entre os conjuntos.
- **Ponderamento :** Um grafo é ponderado se em cada aresta possuir um valor numérico associado, chamado de peso ou custo. E não ponderado se não possuir esse valor nas arestas.

- **Conectividade:** Conectividade de um grafo G é um conceito fundamental que mede o grau de interligação entre os vértices de G . Um grafo **não direcionado** é conexo se existe um caminho entre qualquer par de vértices, caso contrário, é desconexo. Já para digrafos, pode ser classificado como **Fortemente Conexos** se, para cada par de vértices u e v , existe um caminho direcionado de u para v e de v para u . Ou **Fracamente Conexos** se existe um caminho entre cada par de vértices, mas sem todas as direções estarem inclusas nas arestas, ou **Desconexo** pelo mesmo motivo dos **não orientados**.

1.3. Algoritmos de Busca

Na teoria dos grafos, Busca em Largura (BFS) e Busca em Profundidade (DFS) são dois algoritmos fundamentais para percorrer ou explorar grafos de forma sistemática, visitando vértices e arestas a partir de um vértice inicial. Esses algoritmos são a base para a resolução de diversos problemas, como a detecção de ciclos, a verificação de conectividade, a ordenação topológica e a busca pelo caminho mais curto em grafos não ponderados.

1.3.1. BFS

A busca em largura (do inglês breadth-first search) é assim chamada porque ela expande a fronteira entre vértices descobertos e não descobertos uniformemente por meio da largura da fronteira, como se fossem círculos concêntricos gerados por uma pedra que se deixa cair em uma superfície de água completamente parada. A busca é feita da seguinte forma:

- A Busca em largura é um dos algoritmos mais simples para se pesquisar em um grafo.
- Dado um grafo $G = (V, E)$ e um vértice de origem distinta s , a busca em largura explora sistematicamente as arestas de G até "descobrir" cada vértice acessível a partir de s .
- O algoritmo calcula a distância (menor número de arestas) desde s até todos os vértices acessíveis desse tipo.
- O algoritmo produz uma "árvore primeiro na extensão" com raiz s que contém todos os vértices acessíveis.
- Para qualquer vértice v acessível a partir de s , o caminho na árvore primeiro na extensão de s até v corresponde a um "caminho mais curto" de s até v em G .
- Descobre todos os vértices à distância k a partir de s , antes de descobrir quaisquer vértices à distância $k + 1$.

A complexidade do BFS é $O(n + m)$, segue abaixo o pseudocódigo para sua execução:

Algoritmo 1: BFS(G, s)

```
1 início
2   para cada vértice  $u \in V(G) - s$  faça
3      $cor[u] := BRANCO$ 
4      $d[u] := \infty \quad \pi[u] := NULL;$ 
5   fim
6    $cor[s] := CINZA;$ 
7    $d[s] := 0;$ 
8    $\pi[s] := NULL;$ 
9    $Q := \emptyset;$ 
10   $Emfileira(Q, s);$ 
11   $BFSVisit;$ 
12 fim
```

Algoritmo 2: BFSVisit

```
1 início
2   enquanto  $Q \neq \emptyset$  faça
3      $u := desemfila(Q);$ 
4     para  $v \in Adj[u]$  faça
5       se ( $cor[v] = BRANCO$ ) então
6          $cor[v] := CINZA;$ 
7          $d[v] := d[u] + 1;$ 
8          $\pi[v] := u;$ 
9          $Emfileira(Q, v);$ 
10    fim
11    fim
12     $cor[u] := PRETO;$ 
13  fim
14 fim
```

1.3.2. DFS

A busca em profundidade (do inglês depth-first search) é um algoritmo para caminhar no grafo. A estratégia seguida pelo algoritmo é a de buscar, sempre que possível, o mais profundo no grafo. A busca é feita da seguinte forma:

- As arestas são exploradas a partir do vértice v mais recentemente descoberto que ainda tem arestas inexploradas saindo dele.
- Quando todas as arestas de v são exploradas, a busca regressa para explorar as arestas que deixam o vértice a partir do qual v foi descoberto.
- Esse processo continua até descobrirmos todos os vértices acessíveis a partir do vértice de origem inicial.
- Se restarem quaisquer vértices não descobertos, então um deles será selecionado como uma nova origem, e a busca se repetirá a partir daquela origem.
- Esse processo inteiro será repetido até que todos os vértices seja descobertos.

O DFS tem complexidade $O(|V| + |E|)$, segue abaixo o pseudocódigo para a sua execução:

Algoritmo 3: DFS(G)

```
1 início
2   para cada  $u \in V(G)$  faça
3      $cor[u] := BRANCO$ ;
4      $\pi[u] := NULL$ ;
5   fim
6    $tempo := 0$ ;
7   para cada  $u \in V(G)$  faça
8     se  $cor[u] = BRANCO$  então
9        $DFSVisit(G, u)$ ;
10    fim
11  fim
12 fim
```

Algoritmo 4: DFSVisit(G, u)

```
1 início
2    $tempo := tempo + 1$ ;
3    $d[u] = tempo$ ;
4    $cor[u] = CINZENTO$ ;
5   para cada  $v \in Adj[u]$  faça
6     se  $(cor[v] = BRANCO)$  então
7        $\pi[v] := u$ ;
8        $DFSVisit(G, v)$ ;
9     fim
10  fim
11   $cor[u] := PRETO$ ;
12   $tempo := tempo + 1$ ;
13   $f[u] := tempo$ ;
14 fim
```

2. Desenvolvimento

Após a apresentação dos conceitos teóricos, o foco agora volta-se para a estrutura do programa em si. Serão abordadas as tecnologias utilizadas no desenvolvimento, as principais funções da interface e a implementação prática dos algoritmos BFS e DFS.

2.1. Tecnologias utilizadas

Para a sua confecção, optou-se pela utilização da linguagem Python na versão 3.11.9, devido à sua vasta gama de bibliotecas voltadas para análise de dados e interfaces web. As principais tecnologias empregadas foram o **NetworkX** e o **Dash**.

A biblioteca NetworkX atua no *backend* da aplicação, sendo responsável por toda a estruturação lógica do grafo. Ela gerencia a adição e remoção de vértices e arestas, o

cálculo de graus, as classificações estruturais (como conectividade e direcionamento) e a execução estruturada dos algoritmos de busca.

Por outro lado, o Dash foi utilizado no *frontend* para renderizar a interface gráfica interativa, em conjunto com os seus principais componentes core o *Dash Cytoscape* e alguns componentes adicionais para estilização o *Dash Bootstrap Components*. Permitindo que o usuário manipule o grafo diretamente no navegador web.

2.2. Interface

A interface do programa foi dividida em um painel de controle e uma área de visualização principal com alguns botões presentes no cabeçalho da página, cada um com a sua respectiva função. O painel de controle lateral é retrátil, ou seja, durante a visualização ele pode ser ocultado para que o usuário veja apenas o que está na visualização. Através dele é possível inserir novos vértices, trocar o modo de execução atual, alternar a configuração do grafo atual para visualizar grafos orientados ou não orientados, ponderados ou não ponderados, e ainda permite a seleção e acionamento de um dos algoritmos de busca disponíveis.

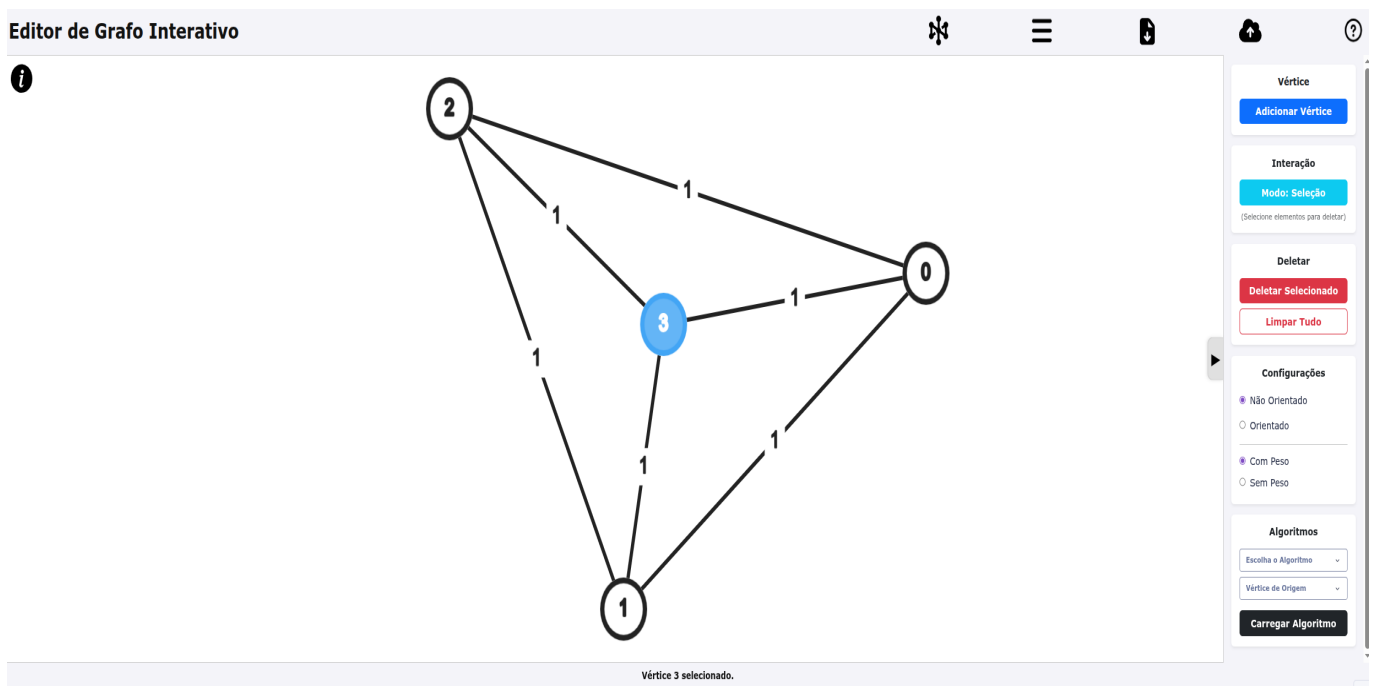


Figura 1. Interface principal do visualizador com o painel lateral ativo.

2.3. Funcionalidades

As principais funcionalidades implementadas nesse painel de visualização consistem na seguinte lista de funções:

- Modos de Interação: um para **Seleção**, onde é possível selecionar e deletar elementos, e um para **Conexão**, que permite a criação de arestas no grafo caso o usuário clique em dois vértices distintos, ou duas vezes no mesmo vértice, criando assim um laço;

- Alternar visualização de grafos entre orientados e não orientados;
- Alternar visualização de grafos entre ponderados e não ponderados;
- Adicionar/remover vértices e arestas de forma interativa;
- Editar peso de arestas e rótulos de vértices;
- Carregar e salvar grafos em formato de lista de adjacências;
- Função para limpar a tela;
- Função para redefinir as posições dos vértices;
- Execução dos algoritmos DFS e BFS com animações e controle;
- Painel com modelos prontos de grafos para carregar na tela;
- Mini cartão com informações dinâmicas sobre o grafo atual.

No cabeçalho também contamos com algumas funções exclusivas que permitem maior interação sobre a área de visualização do grafo, são elas da esquerda para a direita, veja a Figura 2:

- **Redefinição:** Permite ao usuário redefinir as posições dos vértices em formato circular;
- **Modelos:** Uma lista de modelos de grafos pré-montados com diferentes configurações, permite o usuário carregar algum modelo de sua escolha limpando a tela atual no processo;
- **Download:** Função que baixa um arquivo no formato TXT referente a lista de adjacências do grafo atual;
- **Upload:** Função que permite ao usuário subir os grafos de sua escolha, caso o arquivo txt esteja de acordo e sem erros;
- **Ajuda:** Abre um modal de ajuda com um guia de todos os atalhos implementados na plataforma.

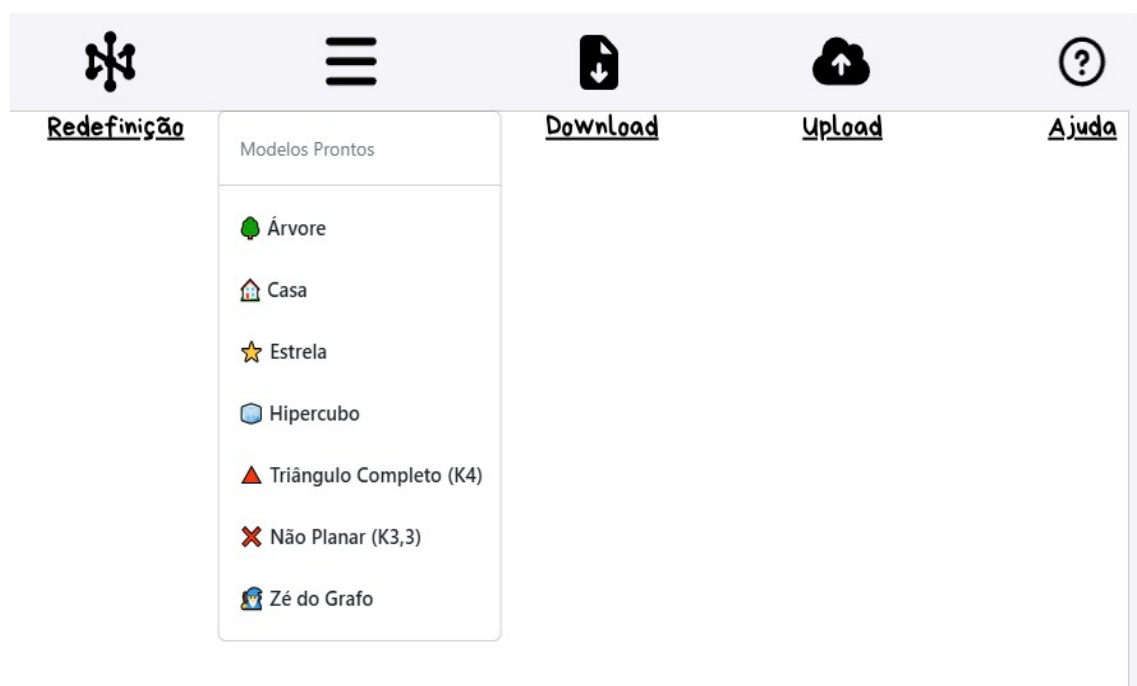


Figura 2. Cabeçalho e suas funções demarcadas.

Alguns atalhos foram implementados para permitir a interação do usuário direto com a área de visualização do grafo sem a necessidade de usar as funções do painel lateral. Para implementar esses atalhos, foram feitos scripts em JavaScript, onde é possível fazer uma ponte de interação direta entre o navegador com o programa em Python, dessa forma chamando algumas funções internas já existentes e outras exclusivas desses atalhos. Para acessar o guia com todos os atalhos e suas funções basta clicar no botão do cabeçalho referente a "Ajuda", dessa forma abrindo o seguinte modal com as instruções:



Figura 3. Guia de atalhos.

Além disso, na área de visualização no canto superior esquerdo, veja a Figura 4, está a função do mini cartão dinâmico de informações sobre o grafo. Ao clicar nele, um cartão com as informações gerais sobre o grafo será revelado e ao clicar sobre algum elemento, seja ele um vértice ou aresta, o mini cartão será expandido com algumas informações adicionais sobre o elemento selecionado.

Para ser dinâmico, todas essas informações estão sendo calculadas em tempo real e acessíveis somente por meio deste botão, ou seja, após o usuário de alguma forma atualizar o conteúdo da visualização, o conteúdo do cartão também será atualizado. Todas essas informações são calculadas respeitando a base teórica apresentada nas Seções 1.1 e 1.2 deste resumo.

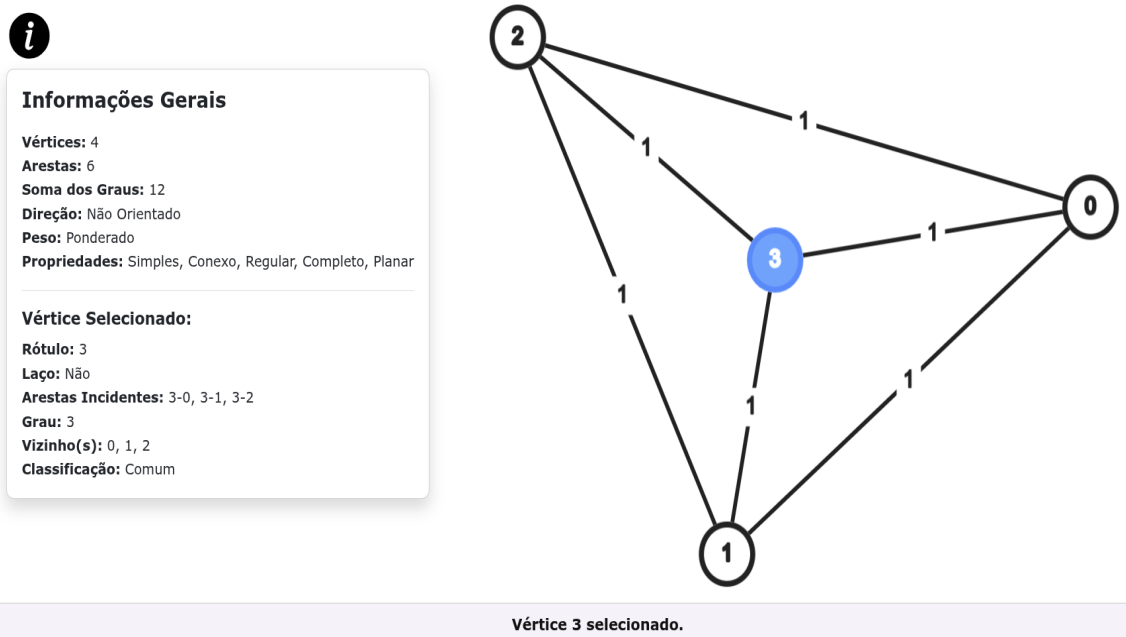


Figura 4. Mini cartão com as informações dinâmicas.

2.4. Algoritmos de Busca

Através da interação com a última das funções do painel lateral, visualizador de grafos consegue executar os algoritmos BFS e DFS utilizando uma estratégia de persistência de estados. A cada vértice visitado, o script salva informações relevantes em um *payload* apelidado de **Snapshots**. Ao término da execução, o script retorna apenas esse *payload* para a plataforma do *Dash* que acessa esses dados para mapear e retornar à tela de visualização cada etapa específica do algoritmo de forma organizada.

2.4.1. Implementação do BFS com Snapshots

```
1 def bfs_snapshots(G, s):
2     s = str(s)
3     c = {}
4     d = {}
5     pi = {}
6     snapshots = []
7
8     for u in G.nodes():
9         c[u] = "Branco"
10        d[u] = None
11        pi[u] = None
12
13    nodes_to_visit = list(G.nodes())
14    if s in nodes_to_visit:
15        nodes_to_visit.remove(s)
16        nodes_to_visit.insert(0, s)
17
18    for start_node in nodes_to_visit:
19        if c[start_node] == "Branco":
```

```

20
21     c[start_node] = "Cinza"
22     d[start_node] = 0
23     pi[start_node] = None
24     Q = [start_node]
25
26     tipo_acao = 'Inicio' if start_node == s else 'Nova_Arvore'
27     msg_inicio = f"Iniciando Busca em Largura (BFS) a partir do
    vértice {start_node}." if start_node == s else f"Iniciando nova á
    rvore de busca a partir do vértice {start_node}."
28
29     snapshots.append({
30         'acao': tipo_acao,
31         'u': start_node,
32         'c': c.copy(),
33         'd': d.copy(),
34         'pi': pi.copy(),
35         'Q': list(Q),
36         'descricao': msg_inicio
37     })
38
39     while Q:
40         u = Q.pop(0)
41
42         snapshots.append({
43             'acao': 'Processando',
44             'u': u,
45             'c': c.copy(),
46             'd': d.copy(),
47             'pi': pi.copy(),
48             'Q': list(Q),
49             'descricao': f"Explorando os vizinhos do vértice {u
    }..."
50
51         })
52
53         for v in G.adj[u]:
54             if c[v] == "Branco":
55                 c[v] = "Cinza"
56                 d[v] = d[u] + 1
57                 pi[v] = u
58                 Q.append(v)
59
60                 snapshots.append({
61                     'acao': 'Descobrendo', 'u': u,
62                     'v': v,
63                     'aresta_atual': (u, v),
64                     'c': c.copy(),
65                     'd': d.copy(),
66                     'pi': pi.copy(),
67                     'Q': list(Q),
68                     'descricao': f"Vértice {v} descoberto!
    Adicionado à fila."
69
70                 })
71             else:
72                 snapshots.append({
73                     'acao': 'Ignorando',

```

```

72         'u': u,
73         'v': v,
74         'aresta_atual': (u, v),
75         'c': c.copy(),
76         'd': d.copy(),
77         'pi': pi.copy(),
78         'Q': list(Q),
79         'descricao': f"Aresta ignorada: Vértice {v}
já foi descoberto."
80     })
81
82     c[u] = "Preto"
83
84     snapshots.append({
85         'acao': 'Finalizando',
86         'u': u,
87         'c': c.copy(),
88         'd': d.copy(),
89         'pi': pi.copy(),
90         'Q': list(Q),
91         'descricao': f"Vértice {u} totalmente explorado."
92     })
93
94     return snapshots

```

Listing 1. Floresta BFS com Snapshots.

2.4.2. Implementação do DFS com Snapshots

```

1 def dfs_snapshots(G, source=None):
2     c = {}
3     pi = {}
4     d = {}
5     f = {}
6     snapshots = []
7     pilha = []
8
9     for u in G.nodes():
10         c[u] = "Branco"
11         pi[u] = None
12         d[u] = None
13         f[u] = None
14
15     tempo = 0
16
17     def dfs_visit(u):
18         nonlocal tempo
19         pilha.append(u)
20         tempo += 1
21         d[u] = tempo
22         c[u] = "Cinza"
23
24         snapshots.append({
25             'acao': 'Descobrendo',

```

```

26         'u': u,
27         'c': c.copy(),
28         'pi': pi.copy(),
29         'd': d.copy(),
30         'f': f.copy(),
31         'pilha': pilha.copy(),
32         'tempo': tempo,
33         'descricao': f"Descobriu o vértice {u} no tempo {tempo}."
34     })
35
36     for v in G.adj[u]:
37         if c[v] == "Branco":
38             pi[v] = u
39
40             snapshots.append({
41                 'acao': 'Avançando',
42                 'u': u,
43                 'v': v,
44                 'aresta_atual': (u, v),
45                 'c': c.copy(),
46                 'pi': pi.copy(),
47                 'd': d.copy(),
48                 'f': f.copy(),
49                 'pilha': pilha.copy(),
50                 'tempo': tempo,
51                 'descricao': f"Avançando do vértice {u} para o
52 vizinho {v}."
53             })
54             dfs_visit(v)
55         else:
56             snapshots.append({
57                 'acao': 'Retorno',
58                 'u': u,
59                 'v': v,
60                 'aresta_atual': (u, v),
61                 'c': c.copy(),
62                 'pi': pi.copy(),
63                 'd': d.copy(),
64                 'f': f.copy(),
65                 'pilha': pilha.copy(),
66                 'tempo': tempo,
67                 'descricao': f"Aresta ignorada: Vértice {v} já foi
68 visitado."
69             })
70
71     c[u] = "Preto"
72     tempo += 1
73     f[u] = tempo
74     pilha.pop()
75
76     snapshots.append({
77         'acao': 'Finalizando',
78         'u': u,
79         'c': c.copy(),
80         'pi': pi.copy(),
81         'd': d.copy(),

```

```

80         'f': f.copy(),
81         'pilha': pilha.copy(),
82         'tempo': tempo,
83         'descricao': f"Retornou para {u}. Vizinhos esgotados no
tempo {tempo}."
84     })
85
86     nodes_to_visit = list(G.nodes())
87     if source is not None and str(source) in nodes_to_visit:
88         nodes_to_visit.remove(str(source))
89         nodes_to_visit.insert(0, str(source))
90
91     for u in nodes_to_visit:
92         if c[u] == "Branco":
93             snapshots.append({
94                 'acao': 'Nova_Arvore',
95                 'u': u,
96                 'c': c.copy(),
97                 'pi': pi.copy(),
98                 'd': d.copy(),
99                 'f': f.copy(),
100                 'pilha': pilha.copy(),
101                 'tempo': tempo,
102                 'descricao': f"Iniciando nova árvore de busca a partir
de {u}."
103             })
104             dfs_visit(u)
105
106     return snapshots

```

Listing 2. Floresta DFS com Snapshots.

2.4.3. Execução dos algoritmos no painel

Para executar um dos algoritmos, o usuário terá que selecioná-lo na aba **Algoritmos** do painel lateral, junto com um vértice de origem para começar a busca no grafo atual. Após isso, o modo de execução será ativado revelando um miniplayer com controles básicos da animação, sendo eles *Play/Pause*, *Foward*, *Fast-Foward*, *Backward*, *Fast-Backward* e *Stop* para parar animação e sair do modo de execução. Juntos deles, logo acima do miniplayer, o painel também irá revelar uma tabela com o passo a passo de execução, utilizando as informações que foram guardadas em **Snapshots**. Veja as Figuras 5 e 6, aonde mostramos o modo de execução do visualizador.

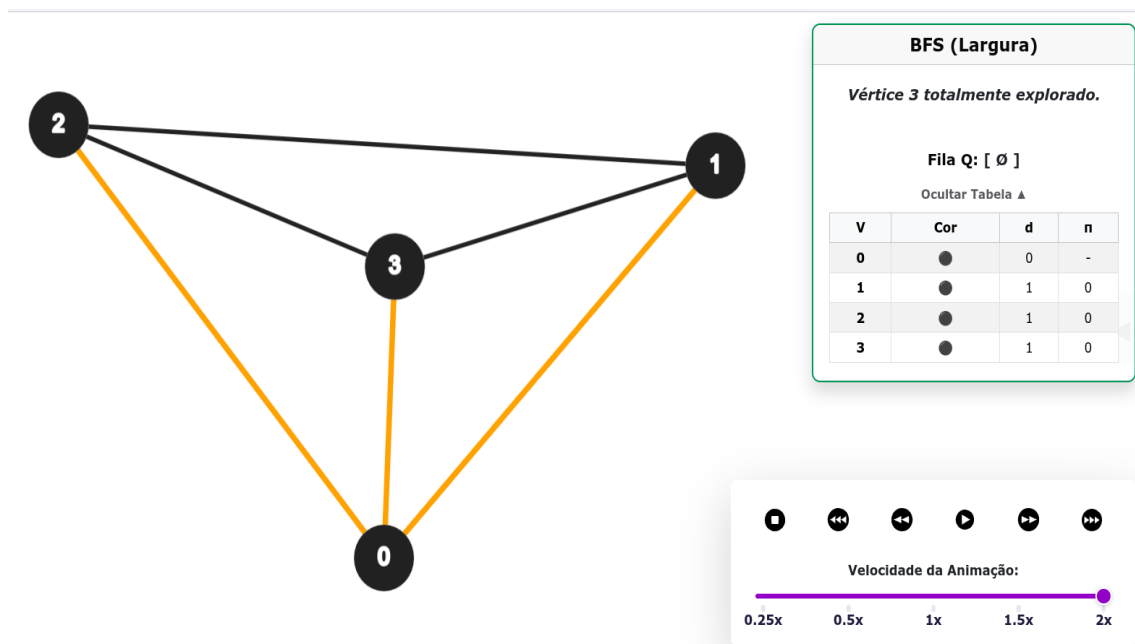


Figura 5. Execução do BFS no visualizador junto da tabela execução.

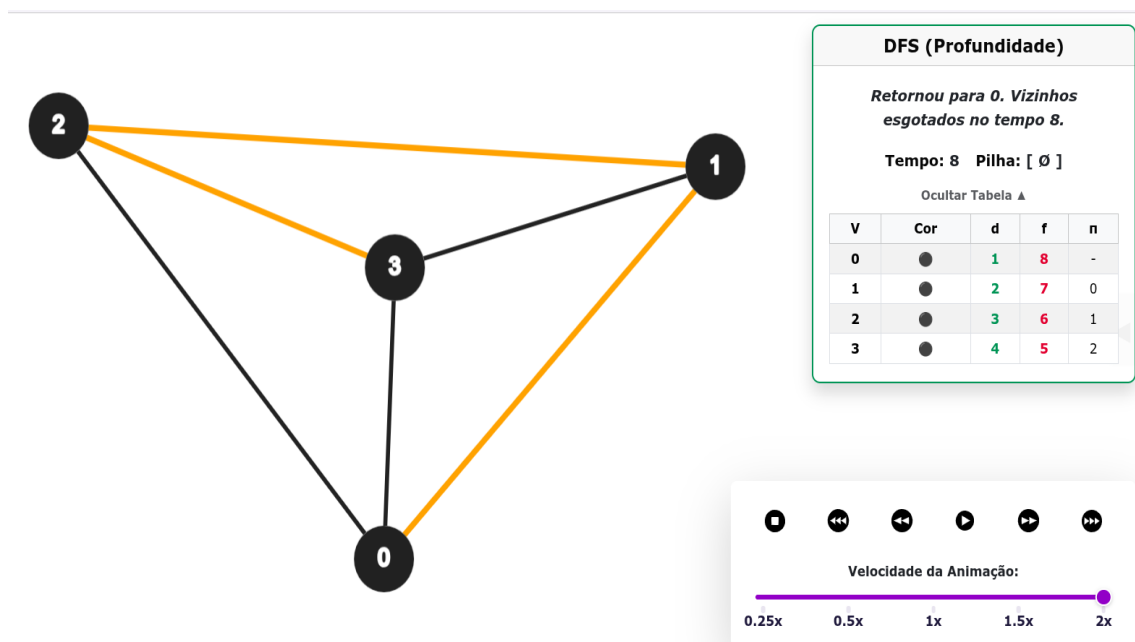


Figura 6. Execução do DFS no visualizador junto da tabela execução.

3. Componentes Fortemente Conexas (SCC)

Nesta seção, abordaremos o tema central sorteado para a nossa apresentação: as Componentes Fortemente Conexas (do inglês, *Strongly Connected Components* - SCC). O objetivo principal desta discussão é definir formalmente o que são SCCs, detalhar suas propriedades matemáticas subjacentes, compreender a motivação algorítmica por trás da sua identificação e citar algumas de suas principais aplicações práticas. Por fim, exploraremos

os principais algoritmos utilizados para encontrá-las, com destaque para os métodos de **Tarjan** e **Kosaraju**, seguidos pela implementação prática no nosso visualizador.

3.1. Definição e Propriedades

Seja um digrafo (grafo direcionado) $G = (V, E)$. Uma Componente Fortemente Conexa de G é um **subgrafo maximal** no qual, para qualquer par de vértices u e v , existe um caminho direcionado de u para v e outro de v para u . Em outras palavras, todos os nós dentro de uma **SCC** são mutuamente alcançáveis.

A conectividade forte confere as seguintes propriedades estruturais estritas para a organização matemática do grafo:

- **Relação de Equivalência:** O alcance mútuo entre os vértices ($u \sim v$) classifica-se sob três regras fundamentais: **Reflexiva** ($u \sim u$), **Simétrica** (se $u \sim v$, logo $v \sim u$) e **Transitiva** (se $u \sim v$ e $v \sim w$, então $u \sim w$).
- **Disjunção:** Como consequência direta da equivalência matemática, as SCCs particionam o conjunto total de vértices de forma que cada vértice pertence exatamente a **uma única componente**. A **interseção** entre duas SCCs diferentes é sempre **nula**.
- **Maximalidade:** Cada SCC é maximal em seu escopo, ou seja, é o maior subgrafo possível que preserva a propriedade de conectividade mútua da SCC. A tentativa de incluir qualquer vértice externo faz com que essa propriedade de ida e volta seja perdida.

Abaixo, a Figura 8 ilustra um grafo direcionado onde é possível visualizar de forma nítida a partição de componentes maximais isoladas.

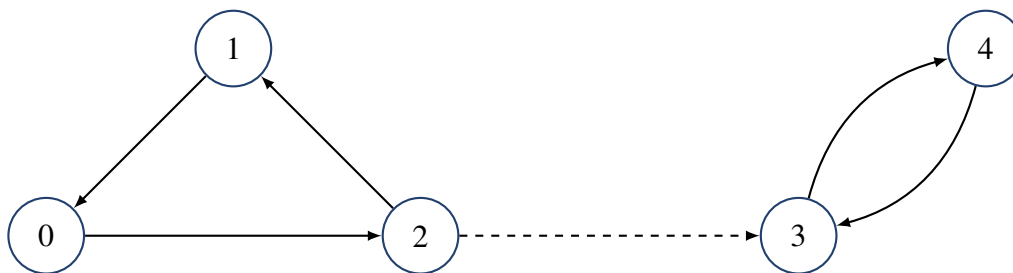


Figura 7. Exemplo ilustrativo de um digrafo com duas SCCs disjuntas: a componente $\{1, 0, 2\}$ e a componente $\{3, 4\}$.

3.2. Motivação do Problema

O estudo e a identificação de SCCs são motivados fundamentalmente pela necessidade de **simplificação estrutural**. Grafos direcionados que modelam problemas reais costumam conter ciclos imprevisíveis, o que torna a análise algorítmica pesada e redundante.

Ao detectarmos as SCCs, podemos executar a chamada **"condensação"**, agrupando todos os vértices de uma mesma componente em um único super-vértice. Ao aplicar esse processo em todas as Componentes do Grafo, isso acaba eliminando todos os ciclos internos e transforma o grafo original complexo em um **Grafo Acíclico Direcionado (DAG)**. O DAG resultante preserva as direções e o fluxo lógico original do sistema,

tornando viável a aplicação imediata de estratégias eficientes, como *Divisão e Conquista* ou *Ordenação Topológica*.

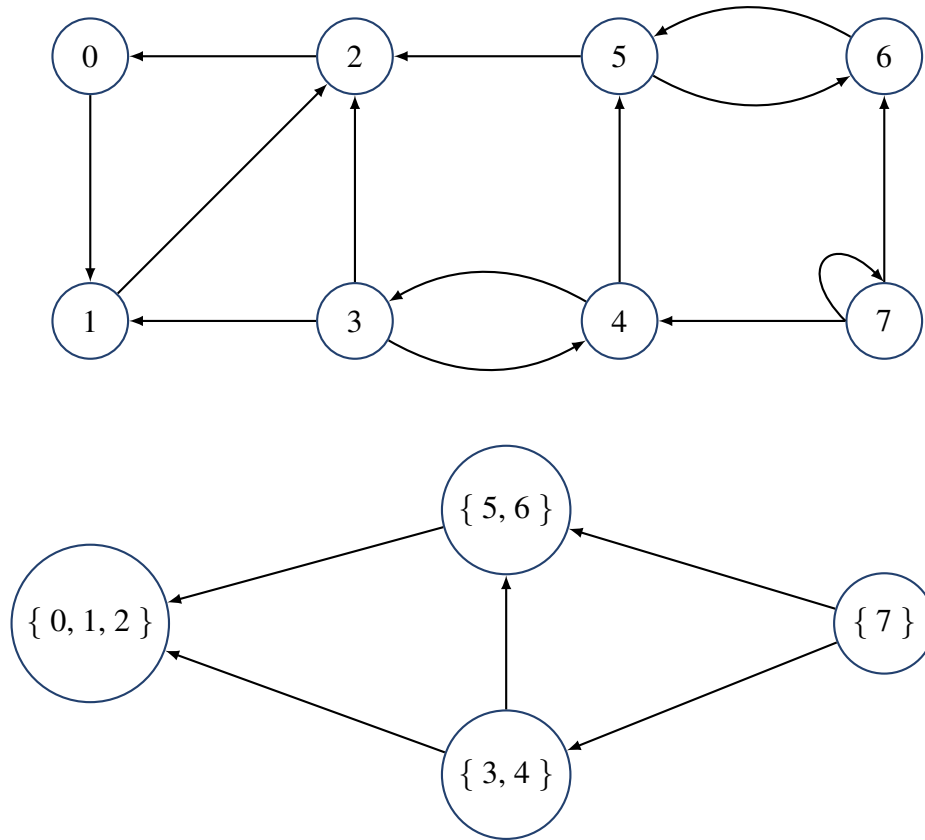


Figura 8. Exemplo ilustrativo de um digrafo com 8 vértices e 4 componentes condensado em um outro digrafo DAG com 4 vértices e 4 componentes disjuntas: $\{0, 1, 2\}$, $\{3, 4\}$, $\{5, 6\}$ e $\{7\}$.

3.3. Aplicações Práticas

Devido a essa alta capacidade de **simplificação lógica** e de **rastreio de ciclos**, algoritmos identificadores de SCC são peças-chave em cenários do mundo real, como:

- **Web Crawling e Redes Sociais:** Identificação de **”bolhas”** altamente conectadas, onde um grupo de pessoas compartilha informações fortemente entre si, ou na identificação de páginas web que interligam os mesmos blocos de assuntos.
- **Algoritmos de recomendação:** Se você entra em um SCC de entusiastas de jogos, o sistema sugere produtos consumidos por outros membros dessa mesma componente.
- **Compiladores e Resolução de Pacotes:** Garantia de estabilidade de software por meio da detecção de dependências cíclicas (quando o Módulo A precisa do B, que precisa do C, que por sua vez bloqueia aguardando o A).
- **Logística e Malhas Rodoviárias:** Análise da viabilidade de trânsito para atestar que, a partir de qualquer ponto dentro de um bairro (subgrafo), um veículo consiga navegar sem encontrar ruas de sentido único que o impeçam de retornar à origem.

3.4. Principais Algoritmos

Para isolar essas componentes de forma veloz e escalável, dois algoritmos clássicos se tornaram o padrão da indústria por garantirem uma complexidade de tempo linear de $O(|V| + |E|)$:

- **Algoritmo de Kosaraju:** Trata-se de um método intuitivo baseado na execução de duas passagens de Busca em Profundidade (DFS). A primeira passagem DFS passa sobre o grafo original, empilhando os vértices de acordo com os seus tempos de término. Em seguida, as direções de todas as arestas do grafo são invertidas, gerando um **Grafo Transposto** G^T no processo. Por fim, uma segunda passagem DFS consome a pilha criada, e cada árvore gerada nessa segunda rodada corresponde perfeitamente a uma SCC disjunta.

Exemplo prático de execução: No grafo da Figura 8, iniciando a primeira DFS pelo vértice 0, os nós do ciclo principal terminam de ser processados primeiro, gerando o topo da pilha com o vértice 7, que encerra a busca por último. Após criar o grafo transposto G^T , a segunda DFS inicia obrigatoriamente por 7, mas devido às inversões das arestas antigas, a busca encontra-se bloqueada contra as direções de entrada de fluxo (contramão), isolando de imediato a componente $\{7\}$. Em seguida, o algoritmo consome o próximo nó do topo da pilha, entrando em $\{3, 4\}$ e repetindo o processo de encurralamento controlado para extrair cada componente restante, logo após consumindo $\{5, 6\}$ e por último $\{0, 1, 2\}$.

- **Algoritmo de Tarjan:** Realiza apenas uma única e eficiente passagem de DFS no grafo original. Durante a busca recursiva, o algoritmo gerencia uma pilha para os nós em exploração e atribui dois valores a cada vértice: um tempo de descoberta (*discovery time*) e um valor de menor retorno alcançável (*lowlink value*). Uma raiz de componente é identificada sempre que o valor de descoberta de um nó se iguala ao seu de *lowlink*.

Exemplo prático de execução: Ao percorrer o ciclo $\{0, 1, 2\}$, o algoritmo atribui a ordem cronológica de descoberta de modo que $d[0] = 1$, $d[1] = 2$ e $d[2] = 3$. Como há uma aresta de retorno de 2 para 0, o valor de menor alcance do nó 2 é propagado recursivamente para trás na pilha, forçando a atualização geométrica de modo que $low[2] = 1$ e $low[1] = 1$. Ao retornar ao ponto de origem, a verificação analisa que $low[0] == d[0] == 1$. O algoritmo reconhece instantaneamente que o vértice 0 é a raiz da componente, executando o desempilhamento atômico em bloco de todos os nós acima dele para capturar a SCC $\{0, 1, 2\}$.

3.5. Implementação no Visualizador

A estratégia de implementação escolhida para o visualizador focou principalmente no **Algoritmo de Tarjan**. A escolha por esse algoritmo se dá por conta da complexidade envolvida em adaptá-lo a plataforma do Dash, enquanto que no de **Kosaraju** seria necessário guardar na memória um outro grafo transposto além de executar uma segunda passagem por todo o grafo para identificar as componentes, no de **Tarjan** a abordagem de uma única passagem DFS evita o custo computacional de transposição de matrizes de adjacência e mantém um resultado mais direto, rápido e fácil visualizar na tabela de execução do algoritmo, por conta disso a estratégia de **Tarjan** que foi adaptada ao visualizador.

Abaixo demonstramos o pseudocódigo que mapeia a busca por profundidade com o controle do *lowlink*:

Algoritmo 5: Tarjan(G)

```

1  início
2       $tempo := 0$ ;
3       $Pilha := \emptyset$ ;
4      para cada  $u \in V(G)$  faça
5           $d[u] := NULL$    $low[u] := NULL$ ;
6           $naPilha[u] := FALSO$ ;
7      fim
8      para cada  $u \in V(G)$  faça
9          se  $d[u] = NULL$  então
10              $TarjanVisit(G, u)$ ;
11         fim
12     fim
13 fim

```

Algoritmo 6: TarjanVisit(G, u)

```

1  início
2       $tempo := tempo + 1$ ;
3       $d[u] := tempo$    $low[u] := tempo$ ;
4       $Empilha(Pilha, u)$ ;
5       $naPilha[u] := VERDADEIRO$ ;
6      para cada  $v \in Adj[u]$  faça
7          se  $d[v] = NULL$  então
8               $TarjanVisit(G, v)$ ;
9               $low[u] := \min(low[u], low[v])$ ;
10         fim
11         senão se  $naPilha[v] = VERDADEIRO$  então
12              $low[u] := \min(low[u], d[v])$ ;
13         fim
14     fim
15     se  $low[u] = d[u]$  então
16          $NovaSCC := \emptyset$ ;
17         Repita
18              $w := Desempilha(Pilha)$ ;
19              $naPilha[w] := FALSO$ ;
20              $Adiciona(NovaSCC, w)$ ;
21         Até  $w = u$ ;
22     fim
23 fim

```

Da mesma forma que os algoritmos de BFS e DFS apresentados na seção anterior, o algoritmo de Tarjan foi adaptado na plataforma Web do Dash para gerar uma lista de estados persistentes (*Snapshots*) a cada iteração de seus laços de execução. Isso possibi-

lita que os recursos de detecção (*lowlink*) e a extração lógica das raízes presentes na pilha sejam totalmente animados em tempo real na plataforma do Dash, e por consequência nos permite preencher a tabela de execução do algoritmo com dados valiosos para o Tarjan. Segue abaixo a implementação feita com a estratégia do Tarjan no script em Python:

```

1 def scc_snapshots(G, s):
2     s = str(s)
3     index = 0
4     stack = []
5     lowlink = {}
6     index_map = {}
7     on_stack = []
8     sccs = []
9     snapshots = []
10
11     def strongconnect(v):
12         nonlocal index
13         index_map[v] = index
14         lowlink[v] = index
15         index += 1
16         stack.append(v)
17         on_stack.append(v)
18         snapshots.append({'acao': 'Visitando', 'v': v,
19                          'index': index_map.copy(), 'lowlink': lowlink.copy(),
20                          'stack': list(stack), 'on_stack': list(on_stack),
21                          'sccs': list(sccs), 'descricao': f"Visitando o vértice {v}."
22                          })
23         for w in G.adj[v]:
24             if w not in index_map:
25                 strongconnect(w)
26                 lowlink[v] = min(lowlink[v], lowlink[w])
27                 snapshots.append({
28                     'acao': 'Atualizando Lowlink', 'v': v, 'w': w,
29                     'index': index_map.copy(), 'lowlink': lowlink.copy()
30                     },
31                     {
32                         'acao': 'Atualizando lowlink de {v} após
33                         visitar {w}.'
34                     })
35             elif w in on_stack:
36                 lowlink[v] = min(lowlink[v], index_map[w])
37                 snapshots.append({
38                     'acao': 'Encontrando Arestas de retorno', 'v': v, 'w': w,
39                     'index': index_map.copy(), 'lowlink': lowlink.copy()
40                     },
41                     {
42                         'acao': 'Encontrada aresta de retorno de {v}
43                         para {w}, atualizando lowlink.'
44                     })
45
46     if lowlink[v] == index_map[v]:
47         scc = []

```

```

46         while True:
47             w = stack.pop()
48             on_stack.remove(w)
49             scc.append(w)
50             if w == v:
51                 break
52         sccs.append(scc)
53         snapshots.append({
54             'acao': 'Formando SCC', 'scc_formada': scc,
55             'index': index_map.copy(), 'lowlink': lowlink.copy(),
56             'stack': list(stack), 'on_stack': list(on_stack),
57             'sccs': list(sccs),
58             'descricao': f"Formada a componente fortemente
conectada: {scc}."
59         })
60         if s in G.nodes() and s not in index_map:
61             strongconnect(s)
62         for node in G.nodes():
63             if node not in index_map:
64                 strongconnect(node)
65         snapshots.append({
66             'acao': 'Finalizado',
67             'index': index_map.copy(),
68             'lowlink': lowlink.copy(),
69             'stack': list(stack),
70             'on_stack': list(on_stack),
71             'sccs': list(sccs),
72             'descricao': f"Algoritmo finalizado! Foram encontradas {len(
sccs)} Componentes Fortemente Conexas (SCCs)."
73         })
74         return snapshots

```

Listing 3. Estrutura base da função de identificação de SCC baseada em Tarjan com captura de *Snapshots*.

Dessa forma conseguimos extrair diversas informações pertinentes à execução do algoritmo e organizá-las na tela no formato de tabela de execução, assim como é feito para o **BFS** e **DFS**. Para o **algoritmo de Tarjan** conseguimos extrair o **conjunto de vértices (V)** analisados, o **tempo de descoberta (d)** de cada vértice pela passagem DFS, se o vértice analisado **está presente ou não na pilha** de execução esperando ser analisado, e por fim a coluna com todos os **lowlink (low)** que são atualizados a cada iteração do algoritmo, por meio dessa coluna conseguimos identificar facilmente também quais vértices pertencem a mesma componente bastando observar se o conjunto em questão possui o mesmo valor **lowlink**, por fim no **cabeçalho** também é indicado a **Maior SCC** e cada **mensagem** descrevendo o passo atual. Além de todas essas informações presentes na tabela, também há o feedback visual ao usuário durante a execução, aonde a cada passo do algoritmo a plataforma do Dash demarcada o caminho escolhido para a passagem DFS, igual é feito no script do próprio DFS, e **sorteia uma cor** para cada **SCC descoberta** no processo, facilitando a visualização de todas as componentes. Veja a Figura 9 com a execução do algoritmo de Tarjan no visualizador.

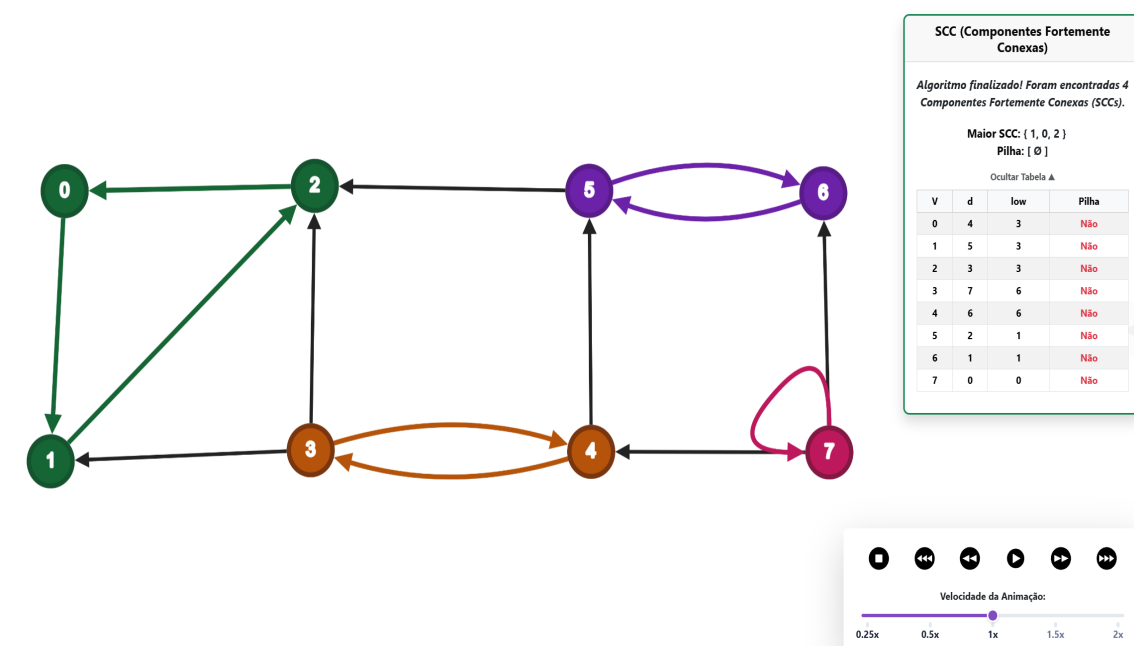


Figura 9. Execução do Algoritmo de Tarjan no visualizador junto da tabela execução.

4. Conclusão

A teoria dos grafos desempenha um papel fundamental na Ciência da Computação, sendo base para diversos algoritmos e aplicações práticas. Neste trabalho foi apresentado o desenvolvimento de um programa para visualização interativa de grafos, com o objetivo de facilitar a compreensão de conceitos teóricos e demonstrar, de forma visual, o funcionamento de algoritmos clássicos de busca.

A aplicação desenvolvida, utilizando Python em conjunto com as bibliotecas NetworkX e Dash, possibilita ao usuário criar, editar e explorar grafos de maneira dinâmica diretamente no navegador. Por meio da interface gráfica interativa, é possível manipular vértices e arestas, alterar propriedades do grafo e executar os algoritmos BFS e DFS com suporte visual, tornando o processo de aprendizado mais intuitivo.

Adicionalmente, a incorporação do estudo e da identificação de Componentes Fortemente Conexas (SCC) elevou o patamar analítico da ferramenta, permitindo ir além dos percursos tradicionais para explorar a topologia de ciclos em digrafos. A implementação bem-sucedida do Algoritmo de Tarjan com suporte a *snapshots* comprovou que é possível rastrear propriedades complexas de conectividade mútua e maximalidade em tempo linear $O(|V| + |E|)$. Desse modo, o visualizador demonstrou de forma clara e prática o conceito de condensação estrutural, evidenciando como sistemas cíclicos complexos podem ser mapeados em Grafos Acíclicos Direcionados (DAG) para viabilizar resoluções lógicas otimizadas em cenários reais.

Dessa forma, o projeto demonstra como a integração entre fundamentos teóricos e ferramentas computacionais pode auxiliar na visualização e no entendimento de estruturas abstratas, evidenciando o potencial de interfaces interativas no ensino e estudo de algoritmos e estruturas de dados baseadas em grafos.

Futuramente, procura-se adicionar mais funcionalidades, otimizações de performance para grafos de grande escala, e mais recursos de QOL (*Quality of Life*) de forma geral, visando proporcionar melhores experiências de usabilidade e interatividade para o usuário.

5. Bibliografia

1. CORMEN, T H. et al. Algoritmos: teoria e prática. Rio de Janeiro: Editora Campus. 3ª Edição. 2012.
2. NETTO, P O B. Grafos - Teorias, Modelos, Algoritmos. Blucher. 5ª Edição. 2012.
3. GOLDBARG, E. et al. Grafos - Conceitos, Algoritmos e Aplicações. Elsevier – Campus. 1ª Edição. 2012.
4. GOMES, P. C. R. Grafos: conceitos fundamentais, algoritmos e aplicações. Editora do IFC. 2022.
5. BONDY, J. A., Murty, U. S. R. Graph Theory. Springer. 2007.
6. FEOFILOFF, P. Algoritmos para Grafos: Componentes fortemente conexas. Instituto de Matemática e Estatística da Universidade de São Paulo (IME-USP). Disponível em: https://www.ime.usp.br/~pf/algoritmos_para_grafos/aulas/strong-comps.html. Acesso em: 12 mai. 2026.
7. SZWARCFITER, J. L. (2018). Teoria Computacional de Grafos: os algoritmos. Elsevier.